ECE 309 Project Report: City Transportation Planner

Team Members: [Name 1], [Name 2], [Name 3]

December 6, 2025

Abstract

This report presents the design, implementation, and validation of a high-performance City Transportation Planner. The system constructs a graph-based representation of a city's transit network to execute efficient shortest-path queries. Core algorithms, including Dijkstra's algorithm and A* search with Euclidean heuristics, were implemented in C++ to ensure optimal performance. Additionally, a prefix-based name lookup system was developed to enhance user interaction. The solution was rigorously optimized for execution speed, utilizing custom data structures and compiler optimizations to minimize query latency. Experimental results demonstrate the correctness of the routing algorithms and the efficiency gains achieved through heuristic search and low-level optimizations.

1 Introduction

The objective of this project is to develop a robust software system capable of modeling city transportation networks and calculating optimal routes between stops. The system processes dataset files to build an adjacency list graph representation and services user queries for shortest paths and stop name lookups. This report details the architectural decisions, algorithmic implementations, and validation results of the developed solution, highlighting the specific contributions of the team and the technical strategies employed to meet performance constraints.

2 Team and Scope

Team Members:

Name 1 ([Email 1]) - [Role]

Name 2 ([Email 2]) - [Role]

Name 3 ([Email 3]) - [Role]

Scope of Work: The team has successfully completed the following components:

x **Required Core**: Data ingestion, Graph construction, Dijkstra's algorithm, and I/O handling.

x **Extension 1**: A* Search algorithm utilizing a Euclidean distance heuristic.

x **Extension 2**: Stop Name Lookup utilizing prefix matching.

x **Bonus 1**: Runtime Optimization for leaderboard competition.

x **Bonus 2**: Simple Route Export to `path.txt`.

3 System Architecture and Implementation

3.1 Graph Representation

The fundamental component of the transportation planner is a memory-efficient graph data structure, implemented in `Graph.cpp`.

- **Adjacency List**: The graph is stored using an adjacency list representation, specifically a `std::vector<std::vector<E` This structure ensures a space complexity of $O(V + E)$, which is optimal for sparse city transportation networks.
- **String Mapping**: To facilitate efficient graph operations, unique string identifiers for stops are mapped to a compact integer range $[0, V-1]$. This mapping is managed via a `std::unordered_map<std::string, int>`, providing $O(1)$ average time complexity for lookups during the graph construction phase.
- **Coordinate Generation**: The A* heuristic requires spatial coordinates for each node. As the input dataset does not provide explicit coordinates, the system deterministically generates (x, y) coordinates by hashing the stop names. This approach ensures consistency of the heuristic function across different execution environments.

3.2 Custom Priority Queue (MinHeap)

To optimize the performance of Dijkstra's algorithm, a custom `MinHeap` class was implemented, replacing the standard library's `std::priority_queue`.

- **Decrease-Key Operation**: The standard STL priority queue lacks a direct `decrease-key` operation, typically necessitating a "lazy deletion" strategy where duplicate nodes are inserted. The custom implementation maintains a `pos` array, mapping each vertex ID to its current index in the heap.
- **Complexity Analysis**: This design enables the `decreaseKey` operation to be performed in $O(\log V)$ time. Consequently, the Dijkstra implementation achieves a time complexity of $O(E \log V)$, which is theoretically superior to the $O(E \log E)$ complexity associated with the lazy-deletion approach, particularly in dense graphs.
- **Memory Efficiency**: The `pos` array facilitates $O(1)$ verification of node presence in the heap (`isInMinHeap`), ensuring the heap size is bounded by V.

3.3 Algorithmic Core

3.3.1 Dijkstra's Algorithm

The implementation of Dijkstra's algorithm (`Dijkstra.cpp`) computes the shortest path from a source to a destination node within a graph with non-negative edge weights.

- **Initialization**: Distance vectors are initialized to infinity, and the source node is inserted into the MinHeap with a distance of 0.
- **Relaxation Process**: In each iteration, the node u with the minimum tentative distance is extracted. The algorithm iterates through neighbors v. If $dist[u] + weight(u, v) < dist[v]$, the distance to v is updated. The node v is then either inserted into the heap or its key is decreased if it is already present.
- **Optimization**: An early exit strategy is employed; the algorithm terminates immediately upon extracting the destination node from the heap, as the shortest path property is guaranteed at this stage.

3.3.2 A* Search (Extension)

The planner was extended to include the A* search algorithm (`AStar.cpp`) to enhance query performance.

- **Heuristic Function**: The Euclidean distance between the current node and the destination serves as the heuristic $h(n)$.
- **Cost Function**: The priority queue orders nodes based on the total estimated cost $f(n) = g(n)+h(n)$, where $g(n)$ is the actual cost from the start node.
- **Performance Impact**: By directing the search towards the goal, A* significantly reduces the search space and the number of expanded nodes compared to Dijkstra's algorithm.

3.3.3 Name Lookup (Extension)

A prefix-based name lookup system (`NameLookup.cpp`) was implemented to support user queries.

- **Data Structure**: A sorted `std::vector<std::string>` containing all stop names is maintained. Sorting is performed once during initialization ($O(V \log V)$).
- **Query Processing**: For a query string S, `std::lower_bound` is utilized to perform a binary search, locating the first string not lexicographically smaller than S. This operation operates in $O(L \log V)$ time, where L is the string length. The system then verifies if the candidate string shares the prefix S.

4 Validation and Experimental Results

The implementation was rigorously validated using the provided test cases. The output format adheres strictly to the project specifications: `ok <distance> <eta>` for successful queries and `error no_path` for failures.

4.1 Dijkstra Validation (Required)

The planner was executed with the provided `example3.txt` dataset.

```
Input:
Central_Station Airport_T5
Museum North_Park
Hill_Top Ferry_Pier

Output (stdout):
ok 3275.9 2340
ok 915.4 654
error no_path

Debug (stderr):
Central_Station BusHub Airport_T5
debug: expanded=6 time=2.083e-06s
Museum North_Park
debug: expanded=4 time=1.25e-06s
```

The results confirm that the Dijkstra implementation correctly identifies shortest paths and handles disconnected components as expected.

4.2 A* Performance Analysis

The planner was executed with the `--astar` flag to verify the heuristic search.

```
Output (stdout):
ok 3275.9 2340
ok 915.4 654
error no_path

Debug (stderr):
Central_Station BusHub Airport_T5
debug: expanded=3 time=1.417e-06s
Museum North_Park
debug: expanded=2 time=5.84e-07s
```

Comparative Analysis: The A* algorithm produced identical path results to Dijkstra, verifying its correctness. Crucially, it demonstrated superior efficiency by expanding significantly fewer nodes (3 vs 6 for the first query), validating the effectiveness of the Euclidean heuristic.

4.3 Name Lookup Verification

The prefix matching capability was tested using the `--lookup` flag.

```
Input:
Cent
Bus
Air
Mus
Nor
Fer
Mall
Hill
Zzz

Output:
found Central_Station
found BusHub
found Airport_T5
found Museum
found North_Park
found Ferry_Pier
found Mall_East
found Hill_Top
not_found
```

The lookup module correctly identified the closest lexicographical match for each prefix.

5 Performance Optimization and Benchmarking

Significant engineering effort was directed towards optimizing the runtime performance to compete for the leaderboard bonus. The optimization strategy encompassed algorithmic efficiency, memory management, and compiler optimizations.

5.1 Algorithmic Optimizations

- **Custom Heap Implementation**: As detailed in Section 3.2, the custom `MinHeap` avoids the overhead of the standard library's priority queue. By supporting `decreaseKey`, the heap size is strictly bounded by V. This reduces the height of the heap, making insertion and extraction operations faster ($O(\log V)$) compared to the lazy-deletion approach ($O(\log E)$).
- **Efficient Graph Storage**: Integer IDs are used internally for all graph operations. The string-to-int mapping occurs only once during input parsing. This ensures that the critical path of the algorithm (edge relaxation) involves only fast integer arithmetic and array lookups, avoiding expensive string comparisons or hashing during the search phase.
- **I/O Optimization**: We disabled synchronization between C++ streams and C-style I/O using `std::ios::sync_with_stdio(false)` and untied `cin` from `cout` using `std::cin.tie(nullptr)`. This significantly reduces the overhead of reading large input files and printing results, which is crucial for the strict time limits.

5.2 Compiler and Build Optimizations

- **Optimization Flags**: The project is compiled with the `-O3` flag, enabling aggressive compiler optimizations such as loop unrolling, function inlining, and vectorization.
- **Static Linking**: All source files are compiled into object files and linked statically, allowing the compiler to perform cross-module optimizations.

5.3 Measurement Verification

The provided `runtime_meter.hpp` was utilized to benchmark the code locally. This tool allowed for high-precision profiling (microseconds) of the Dijkstra and A* functions. The debug output confirms that queries execute in the order of microseconds (10^{-6} seconds) on the sample data, well within the 30-second time limit.

6 Bonus: Simple Route Export

We implemented the "Simple Route Export" bonus feature. When the planner is run with the `--export` flag, the sequence of stop names for the found path is written to a file named `path.txt`. This allows for external visualization or verification of the computed routes.

- **Implementation**: A helper function `writePathToFile` was added to `main.cpp`. It takes the reconstructed path (vector of strings) and writes it to the output file stream.
- **Integration**: The export logic is integrated into both the Dijkstra and A* execution flows, ensuring that the path is exported regardless of the algorithm used.

7 Independent Implementation Components

The core algorithmic modules and data structures were designed and implemented entirely by the team members without the assistance of Generative AI or Large Language Models (LLMs). AI tools were utilized solely for the purpose of generating and formatting this technical report. The following components represent our independent work:

1. **Graph.cpp/h**: Implements the adjacency list structure and coordinate handling logic.
2. **MinHeap.cpp/h**: Implements the custom priority queue used in Dijkstra and A*.
3. **Dijkstra.cpp/h**: Implements the core shortest path algorithm.

4. **AStar.cpp/h**: Implements the heuristic search extension.
5. **NameLookup.cpp/h**: Implements the string search extension.

8 Build Instructions and Reproducibility

To reproduce the results presented in this report, the following build and execution steps should be followed:

```
# Navigate to the project directory
cd City-Transportation-Planner

# Compile the project
make

# Execute Dijkstra (Standard Mode)
./planner < tests/example3.txt

# Execute A* Search
./planner --astar < tests/example3.txt

# Execute Name Lookup
./planner --lookup < tests/lookup_queries.txt

# Execute with Route Export (Bonus)
./planner --export < tests/example3.txt
# Check the output file
cat path.txt
```

A Code Implementation Examples

This appendix includes key code snippets from our implementation to demonstrate the algorithmic logic and coding style.

A.1 Graph Construction (Graph.cpp)

```
void Graph::loadFromFile(const std::string& filename) {
    std::ifstream file(filename);
    // ... error handling ...
    std::string line;
    while (std::getline(file, line)) {
        if (line.empty()) continue;
        std::stringstream ss(line);
        std::string u_str, v_str, len_str;
        if (!(ss >> u_str >> v_str >> len_str)) continue;

        double weight = std::stod(len_str); // Simplified parsing

        int u = getId(u_str);
        if (u == -1) {
            u = numVertices++;
            nameToId[u_str] = u;
            idToName.push_back(u_str);
            adj.resize(numVertices);
            coordinates.resize(numVertices);
            generateCoordinates(u, u_str);
        }
        // ... similar logic for v ...
        adj[u].push_back({v, weight});
```

```
        adj[v].push_back({u, weight});
    }
}
```

A.2 MinHeap Decrease Key (MinHeap.cpp)

```
void MinHeap::decreaseKey(int vertex, double newDist) {
    int i = pos[vertex];
    if (i == -1) return;

    heap[i].dist = newDist;

    while (i != 0 && heap[(i - 1) / 2].dist > heap[i].dist) {
        swapNodes(i, (i - 1) / 2);
        i = (i - 1) / 2;
    }
}
```

A.3 Dijkstra's Algorithm (Dijkstra.cpp)

```
PathResult Dijkstra::findShortestPath(const Graph& graph, int src, int dest) {
    // ... initialization ...
    while (!pq.isEmpty()) {
        HeapNode minNode = pq.extractMin();
        int u = minNode.vertex;

        if (u == dest) break; // Early exit

        for (const auto& edge : graph.getNeighbors(u)) {
            int v = edge.to;
            double weight = edge.weight;

            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                parent[v] = u;

                if (pq.isInMinHeap(v)) {
                    pq.decreaseKey(v, dist[v]);
                } else {
                    pq.insert(v, dist[v]);
                }
            }
        }
    }
    // ... path reconstruction ...
}
```

A.4 A* Search (AStar.cpp)

```
// Heuristic calculation is embedded in the priority update
double tentative_g = g_score[u] + weight;

if (tentative_g < g_score[v]) {
    g_score[v] = tentative_g;
    parent[v] = u;
    double f_v = tentative_g + graph.getEuclideanDistance(v, dest);

    if (pq.isInMinHeap(v)) {
        pq.decreaseKey(v, f_v);
    } else {
        pq.insert(v, f_v);
    }
}
```

A.5 Name Lookup (NameLookup.cpp)

```
std::string NameLookup::findClosestMatch(const std::string& query) {
    if (query.empty()) return "";
    auto it = std::lower_bound(sortedNames.begin(), sortedNames.end(), query);

    if (it != sortedNames.end()) {
        if (it->rfind(query, 0) == 0) { // Check prefix
            return *it;
        }
    }
    return "";
}
```